Exam 2 — Sample Questions

W4111 · MongoDB (`classicmodels`) + Neo4j (Movie DB)

> **Based on the professor's guidance:** *"I strongly advise you to understand the `classicmodels` sample data from homework 3 for MongoDB. You are also responsible for having taken the Neo4j tutorial for the Movie Database that was part of homework 3 setup. You are responsible for being able to query the `classicmodels` data and the Neo4j sample movie dataset. WRT to MongoDB, you are responsible for understanding how to use the following aggregation stages/operators:* ***count, group, limit, lookup, match, project, sort, unwind****."*

Each question below is answered **immediately** beneath the prompt, followed by a **brief explanation** of what the code does and why it works.

Table of Contents (with page numbers)

> Tip: the PDF also has a **clickable bookmark outline** (open the "Outline" / "Bookmarks" panel in your reader). The page numbers below are for the printed copy.

Part A — MongoDB (`classicmodels`)

Part B — Neo4j (Movie Database)

Part C — Conceptual / Short-Answer

> **Setup assumed for every MongoDB solution:**
>
> ```
> from pymongo import MongoClient
> client = MongoClient("mongodb://localhost:27017")
> db = client["classicmodels"]
> ```
>
> **Setup assumed for every Neo4j solution:** the sample Movie graph loaded via `:play movies` (labels `Movie`, `Person`; relationships `ACTED_IN`, `DIRECTED`, `PRODUCED`, `WROTE`, `REVIEWED`, `FOLLOWS`).

Part A — MongoDB (`classicmodels`)

A.1 `$match` — filter-only questions

Q1. Simple equality filter on nested field

Prompt. Return every `customers` document whose `address.country` is `"France"`. Return all fields.

Answer.

```
result = list(db["customers"].find({ "address.country": "France" }))
```

Explanation. `find()` takes the equivalent of a `WHERE` clause. Because `address` is an embedded subdocument, we reach into it with **dot notation**: the key `"address.country"` targets the nested `country` field. No projection is given, so every field is returned.

Q2. Compound filter with range + `$in`

Prompt. Return orders whose `status` is one of `"Shipped"` or `"Resolved"` **and** whose `orderDate` falls in 2004 (strings `"2004-01-01"` through `"2004-12-31"`). Return only `orderNumber`, `status`, `orderDate`.

Answer.

```
filt = {
    "status":    { "$in": ["Shipped", "Resolved"] },
    "orderDate": { "$gte": "2004-01-01", "$lte": "2004-12-31" },
}
proj = { "_id": 0, "orderNumber": 1, "status": 1, "orderDate": 1 }
result = list(db["orders"].find(filt, proj))
```

Explanation. `$in` is set-membership (SQL's `IN`). `$gte/$lte` perform ordinary range comparison. In `classicmodels` the dates are stored as **ISO strings**; lexicographic comparison still yields chronological order, so no `$toDate` is needed. The projection sets `_id: 0` to drop the default id.

Q3. Regex / existence filter

Prompt. Return every customer whose `customerName` contains the substring `"Mini"` (case-insensitive) **and** who has a non-null `state`. Return only `customerName` and `address.state`.

Answer.

```
filt = {
    "customerName":  { "$regex": "Mini", "$options": "i" },
    "address.state": { "$ne": None, "$exists": True },
}
proj = { "_id": 0, "customerName": 1, "address.state": 1 }
result = list(db["customers"].find(filt, proj))
```

Explanation. `$regex` with option `"i"` is the case-insensitive substring test. `$exists: true` ensures the field

is present; `$ne: null` filters out rows where it exists but is `null`. Both conditions on the same field are combined by AND when placed inside the same object.

A.2 `$project` — reshape & compute

Q4. Rename, drop `_id`, lift nested fields

Prompt. For every customer return `number` (renamed from `customerNumber`), `name` (renamed from `customerName`), and `country` (lifted from `address.country`). Do not return `_id`.

Answer.

```
pipeline = [
    { "$project": {
        "_id":     0,
        "number":  "$customerNumber",
        "name":    "$customerName",
        "country": "$address.country",
    }}
]
result = list(db["customers"].aggregate(pipeline))
```

Explanation. Inside `$project`, `"newField": "$oldField"` creates a renamed alias. Prefixing a field with `$` tells MongoDB *"the value of this field"*, as opposed to a literal string. Dot notation (`$address.country`) reaches into nested documents.

Q5. Computed column (`$cond` tier label)

Prompt. For every customer return `customerName`, `creditLimit`, and a new field `tier` that is `"gold"` if `creditLimit` > 100 000, `"silver"` if between 50 000 and 100 000 (inclusive), else `"bronze"`.

Answer.

```
pipeline = [
    { "$project": {
        "_id": 0,
        "customerName": 1,
        "creditLimit":  1,
        "tier": {
            "$switch": {
                "branches": [
                    { "case": { "$gt":  ["$creditLimit", 100000] }, "then": "gold"   },
                    { "case": { "$gte": ["$creditLimit",  50000] }, "then": "silver" },
                ],
                "default": "bronze"
            }
        }
    }}
]
result = list(db["customers"].aggregate(pipeline))
```

Explanation. `$switch` is the multi-branch version of `$cond` (a `CASE WHEN` in SQL). Branches are tested top-to-bottom; the first matching `case` wins and its `then` value is returned. If none match, `default` is used.

A.3 `$sort` + `$limit` — Top-N

Q6. Top 10 most expensive line items

Prompt. Return the 10 line items with the largest `priceEach`. Output `orderNumber`, `productCode`, `priceEach`, `quantityOrdered`.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$sort":   { "orderDetails.priceEach": -1 } },
    { "$limit":  10 },
    { "$project": {
        "_id": 0,
        "orderNumber":     1,
        "productCode":     "$orderDetails.productCode",
        "priceEach":       "$orderDetails.priceEach",
        "quantityOrdered": "$orderDetails.quantityOrdered",
    }},
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. Line items live inside the embedded `orderDetails` array, so we first `$unwind` to get one document per line item. `$sort` orders by `priceEach` descending, `$limit` keeps the top 10, and a final `$project` reshapes the output. The `$sort`-before-`$limit` combination is the MQL idiom for "Top-N".

A.4 `$count` — stage vs. accumulator

Q7. Count of documents matching a filter

Prompt. How many customers have `creditLimit >= 100000`?

Answer.

```
pipeline = [
    { "$match": { "creditLimit": { "$gte": 100000 } } },
    { "$count": "nBigCustomers" },
]
result = list(db["customers"].aggregate(pipeline))
# e.g. [ { 'nBigCustomers': 14 } ]
```

Explanation. The `$count` **stage** replaces its input with a single document `{ <fieldName>: <n> }` whose field name is the string you pass. Put `$match` before `$count` so only matching documents are counted.

Q8. `$count` as a stage vs. `$sum: 1` inside `$group`

Prompt. How many orders exist **per status**?

Answer.

```
pipeline = [
    { "$group": { "_id": "$status", "nOrders": { "$sum": 1 } } },
    { "$project": { "_id": 0, "status": "$_id", "nOrders": 1 } },
```

```
    { "$sort": { "nOrders": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. This is the **accumulator** form of counting. Inside `$group`, `{ "$sum": 1 }` adds one for every document that falls into the bucket — the MQL equivalent of `COUNT(*)`. The `$count` stage cannot be used here because we want a per-group count, not a single total.

A.5 `$unwind` — arrays to rows

Q9. Flatten `orderDetails` into line-item documents

Prompt. Produce one document per order line with `orderNumber`, `orderDate`, `productCode`, `quantityOrdered`, `priceEach`. Assert the total count is 2 996.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$project": {
        "_id":             0,
        "orderNumber":     1,
        "orderDate":       1,
        "productCode":     "$orderDetails.productCode",
        "quantityOrdered": "$orderDetails.quantityOrdered",
        "priceEach":       "$orderDetails.priceEach",
    }},
]
result = list(db["orders"].aggregate(pipeline))
assert len(result) == 2996
```

Explanation. `$unwind "$orderDetails"` turns one order with an array of *n* line items into *n* separate documents, each with `orderDetails` pointing at a single element (not an array). Since every order has ≥ 1 line item and HW3 has 326 orders with 2 996 line items total, the assertion succeeds.

Q10. `$unwind` with `preserveNullAndEmptyArrays`

Prompt. Rephrase the above so orders **without** any line items (hypothetically) still survive the pipeline. Explain what changes.

Answer.

```
pipeline = [
    { "$unwind": {
        "path": "$orderDetails",
        "preserveNullAndEmptyArrays": True,
    }},
]
```

Explanation. By default, `$unwind` **drops** any document whose target field is missing, `null`, or an empty array — behaving like an `INNER JOIN`. Setting `preserveNullAndEmptyArrays: true` emits the parent document once with `orderDetails` set to `null`, behaving like a `LEFT OUTER JOIN`. (The `classicmodels` dataset has no empty order arrays, but this is the canonical fix for "why did my pipeline lose rows?")

A.6 `$group` — aggregation

Q11. Revenue per order

Prompt. For each order, return `orderNumber`, `status`, and `totalValue` (sum over its line items of `priceEach * quantityOrdered`). Round to 2 decimals. Sort by `totalValue` descending.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id":    "$orderNumber",
        "status": { "$first": "$status" },
        "totalValue": { "$sum": {
            "$multiply": [ "$orderDetails.priceEach",
                           "$orderDetails.quantityOrdered" ]
        }}
    }},
    { "$project": {
        "_id": 0,
        "orderNumber": "$_id",
        "status": 1,
        "totalValue": { "$round": [ "$totalValue", 2 ] },
    }},
    { "$sort": { "totalValue": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. `$unwind` explodes every order into its line items; `$group` on `orderNumber` then sums `priceEach * quantityOrdered` over each bucket. Because `status` is constant within a bucket, `$first` picks any occurrence of it. The final `$project` renames `_id` back to `orderNumber` and formats the number.

Q12. Distinct customer count per status

Prompt. For each order `status`, compute the number of **distinct customers** who have at least one order with that status.

Answer.

```
pipeline = [
    { "$group": {
        "_id": "$status",
        "customers": { "$addToSet": "$customerNumber" }
    }},
    { "$project": {
        "_id": 0,
        "status":     "$_id",
        "nCustomers": { "$size": "$customers" }
    }},
    { "$sort": { "nCustomers": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. `$addToSet` is the accumulator equivalent of `COUNT(DISTINCT x)` — it collects unique values into an array. `$size` then gives its cardinality. Using `{ $sum: 1 }` here would instead count orders, which

would double-count customers with multiple orders.

Q13. Top 3 best-selling products

Prompt. Return the 3 `productCode` values with the highest total `quantityOrdered` across all orders. Output `productCode`, `totalUnitsSold`.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": "$orderDetails.productCode",
        "totalUnitsSold": { "$sum": "$orderDetails.quantityOrdered" }
    }},
    { "$sort":  { "totalUnitsSold": -1 } },
    { "$limit": 3 },
    { "$project": { "_id": 0,
                    "productCode": "$_id",
                    "totalUnitsSold": 1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. Standard Top-N after aggregation: `$unwind` to reach the line items, `$group` to sum per product, `$sort` + `$limit` for the top 3, `$project` to tidy the keys. Order matters: sorting before grouping would be wrong and would waste work.

A.7 `$lookup` — joins

Q14. Orders with customer name + country

Prompt. For each order return `orderNumber`, `orderDate`, `customerName`, and `country`. Sort by `orderDate` descending; limit 15.

Answer.

```
pipeline = [
    { "$lookup": {
        "from":         "customers",
        "localField":   "customerNumber",
        "foreignField": "customerNumber",
        "as":           "cust"
    }},
    { "$unwind": "$cust" },
    { "$project": {
        "_id":          0,
        "orderNumber":  1,
        "orderDate":    1,
        "customerName": "$cust.customerName",
        "country":      "$cust.address.country",
    }},
    { "$sort":  { "orderDate": -1 } },
    { "$limit": 15 },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. `$lookup` is the equi-join: for each order, it finds every `customers` document whose `customerNumber` equals this order's `customerNumber` and writes the matches into the array field `cust`. Because each order has exactly one customer, `$unwind "$cust"` safely collapses the length-1 array. The final `$project` lifts the needed customer fields.

Q15. Customers without any orders (anti-join)

Prompt. Return `customerNumber` and `customerName` of customers that have **never** placed an order.

Answer.

```
pipeline = [
    { "$lookup": {
        "from":         "orders",
        "localField":   "customerNumber",
        "foreignField": "customerNumber",
        "as":           "theirOrders"
    }},
    { "$match": { "theirOrders": { "$size": 0 } } },
    { "$project": { "_id": 0, "customerNumber": 1, "customerName": 1 } },
    { "$sort":    { "customerNumber": 1 } },
]
result = list(db["customers"].aggregate(pipeline))
```

Explanation. The MQL **anti-join** idiom. After `$lookup`, customers with no orders have an empty `theirOrders` array; `$match` with `$size: 0` keeps only those. This is the MongoDB analog of SQL's `LEFT JOIN ... WHERE right_side IS NULL` or `NOT EXISTS`.

A.8 Kitchen-sink pipelines

Q16. Top 5 customers by revenue (all 8 operators)

Prompt. Return the 5 customers with the highest total revenue. Output `customerNumber`, `customerName`, `country`, `totalRevenue`, `nOrders` (count of distinct orders). Also report — as a separate query — how many customers have **any** revenue at all.

Answer — Top 5 pipeline.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": "$customerNumber",
        "totalRevenue": { "$sum": { "$multiply": [
            "$orderDetails.priceEach", "$orderDetails.quantityOrdered" ] } },
        "orders": { "$addToSet": "$orderNumber" }
    }},
    { "$lookup": {
        "from": "customers",
        "localField": "_id",
        "foreignField": "customerNumber",
        "as": "cust"
    }},
    { "$unwind": "$cust" },
    { "$match": { "cust.address.country": { "$exists": True } } },
    { "$project": {
        "_id": 0,
```

```
        "customerNumber": "$_id",
        "customerName":   "$cust.customerName",
        "country":        "$cust.address.country",
        "totalRevenue":   { "$round": [ "$totalRevenue", 2 ] },
        "nOrders":        { "$size": "$orders" },
    }},
    { "$sort":  { "totalRevenue": -1 } },
    { "$limit": 5 },
]
top5 = list(db["orders"].aggregate(pipeline))
```

Answer — count of customers with any revenue.

```
count_pipeline = [
    { "$group": { "_id": "$customerNumber" } },
    { "$count": "nCustomersWithOrders" },
]
n = list(db["orders"].aggregate(count_pipeline))
```

Explanation. The first pipeline is the canonical "kitchen-sink" query and hits every required stage: `$unwind`, `$group`, `$lookup`, `$match`, `$project`, `$sort`, `$limit`. The second pipeline shows the terminal `$count` stage used after a `$group` that bucketizes by customer — it counts **buckets**, not documents.

Q17. Revenue by country (two `$group`s + `$lookup`)

Prompt. For each country, return `country` and `totalRevenue`. Sort descending.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": "$customerNumber",
        "rev": { "$sum": { "$multiply": [
            "$orderDetails.priceEach", "$orderDetails.quantityOrdered" ] } }
    }},
    { "$lookup": {
        "from": "customers",
        "localField": "_id",
        "foreignField": "customerNumber",
        "as": "cust"
    }},
    { "$unwind": "$cust" },
    { "$group": {
        "_id": "$cust.address.country",
        "totalRevenue": { "$sum": "$rev" }
    }},
    { "$project": {
        "_id": 0,
        "country":      "$_id",
        "totalRevenue": { "$round": [ "$totalRevenue", 2 ] }
    }},
    { "$sort": { "totalRevenue": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. Using **two `$group` stages in a single pipeline** is a common MQL pattern: first reduce to the customer level, join the country in via `$lookup`, then re-aggregate by country. The alternative of grouping directly on the joined country in one pass is also valid but can be harder to read.

Q18. Argmax per group — biggest order per status

Prompt. For each `status`, return the single order with the largest `totalValue`. Output `status`, `orderNumber`, `totalValue`.

Answer.

```
pipeline = [
    { "$unwind": "$orderDetails" },
    { "$group": {
        "_id": { "status": "$status", "orderNumber": "$orderNumber" },
        "totalValue": { "$sum": { "$multiply": [
            "$orderDetails.priceEach", "$orderDetails.quantityOrdered" ] } }
    }},
    { "$sort":  { "_id.status": 1, "totalValue": -1 } },
    { "$group": {
        "_id":         "$_id.status",
        "orderNumber": { "$first": "$_id.orderNumber" },
        "totalValue":  { "$first": "$totalValue" }
    }},
    { "$project": {
        "_id": 0,
        "status":      "$_id",
        "orderNumber": 1,
        "totalValue":  { "$round": [ "$totalValue", 2 ] }
    }},
    { "$sort": { "totalValue": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. This is the MQL **argmax-per-group** idiom: group with a compound `_id` to get per-`(status, order)` totals, **sort so the row with the largest value appears first inside each group**, then re-group by status alone using `$first` to take the winner. Equivalent to SQL's `ROW_NUMBER() OVER (PARTITION BY status ORDER BY totalValue DESC) = 1`.

Part B — Neo4j (Movie Database)

B.1 Pattern matching, triangles, and aggregation

Q19. Return every actor and their movies

Prompt. List every actor's name and the title of each movie they acted in. Sort by actor, then by movie.

Answer.

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name AS actor, m.title AS movie
ORDER BY actor, movie;
```

Explanation. A basic directed pattern: for every `Person` who has an outgoing `ACTED_IN` edge to a `Movie`, return the pair. `RETURN ... AS alias` renames the columns. Implicit grouping does **not** happen here because we aren't calling any aggregate — we get one row per `(actor, movie)` edge.

Q20. Directors who also acted in the same movie

Prompt. Return the names of people who **both** directed and acted in the same movie, together with the movie title.

Answer.

```
MATCH (p:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(p)
RETURN DISTINCT p.name AS name, m.title AS movie
ORDER BY name, movie;
```

Explanation. Reusing the variable `p` on both sides of the pattern forces the director and actor nodes to be identical. The arrow directions (`-[:DIRECTED]->` on the left, `<-[:ACTED_IN]-` on the right) both point **into** the movie, consistent with the schema where all movie-related relationships start at a `Person`. `DISTINCT` de-duplicates in case a person directed and acted in multiple movies.

Q21. Co-actors of Tom Hanks

Prompt. Return the distinct names of all people who acted in a movie with **Tom Hanks** (excluding Tom himself), together with the movie title.

Answer.

```
MATCH (tom:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
      <-[:ACTED_IN]-(co:Person)
WHERE co <> tom
RETURN DISTINCT co.name AS coactor, m.title AS movie
ORDER BY coactor, movie;
```

Explanation. Classic "co-star" triangle: two `ACTED_IN` edges meet at the same `(m:Movie)` node. `WHERE co <> tom` excludes the trivial self-match where `co` would otherwise equal Tom.

Q22. Aggregation with `HAVING`-style filter

Prompt. For each actor, return the number of movies they have acted in. Keep only actors with ≥ 3 movies. Order by count descending, then by name.

Answer.

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WITH p, count(m) AS nFilms
WHERE nFilms >= 3
RETURN p.name AS actor, nFilms
ORDER BY nFilms DESC, actor;
```

Explanation. Cypher's grouping is **implicit**: any non-aggregated variable in a projection (here `p`) is a grouping key. `WITH` acts like an intermediate `SELECT` that lets you filter **on the aggregate** (SQL's `HAVING`). A plain `WHERE` before the `count` wouldn't have access to `nFilms`.

Q23. Filter on a relationship property (`roles`)

Prompt. Find every actor who has ever played the role `"Neo"`, along with the movie in which they did.

Answer.

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE 'Neo' IN r.roles
RETURN p.name AS actor, m.title AS movie;
```

Explanation. `roles` is a property **on the `ACTED_IN` relationship**, not on either node. Binding the relationship to the variable `r` lets us reference `r.roles`. `IN` performs list-membership testing.

Q24. Unwind a relationship list

Prompt. For each `ACTED_IN` relationship, produce one row per role. Output `actor`, `movie`, `role`.

Answer.

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
UNWIND r.roles AS role
RETURN p.name AS actor, m.title AS movie, role
ORDER BY actor, movie, role;
```

Explanation. `UNWIND` is the Cypher equivalent of MongoDB's `$unwind`: it flattens a list into rows. If the list is empty or `null`, `UNWIND` drops the row by default (same behavior as `$unwind`).

Q25. Shortest path (Bacon number)

Prompt. Find the shortest path between Tom Hanks and Kevin Bacon using any relationships. Return the path and its length.

Answer.

```
MATCH p = shortestPath(
  (t:Person {name: 'Tom Hanks'})-[*]-(k:Person {name: 'Kevin Bacon'})
)
RETURN p, length(p) AS baconNumber;
```

Explanation. `shortestPath(...)` wraps a variable-length pattern. `-[*]-` matches any relationship type, in any direction, any number of hops. The result `p` is a path object whose edge count is `length(p)` — the Bacon number.

Q26. Multi-hop: followers-of-followers

Prompt. For each person in the graph, return the names of all people reachable via **exactly 2** `FOLLOWS` hops. Output `person`, `fof` (friend-of-friend). Exclude the person themselves.

Answer.

```
MATCH (a:Person)-[:FOLLOWS*2]->(b:Person)
WHERE a <> b
RETURN DISTINCT a.name AS person, b.name AS fof
ORDER BY person, fof;
```

Explanation. `-[:FOLLOWS*2]->` is a **variable-length** relationship requiring exactly 2 hops of `FOLLOWS`. `a <> b` excludes cycles back to the starting node. `DISTINCT` de-duplicates in case multiple length-2 paths exist between the same pair.

Q27. Directors sorted by average review rating

Prompt. For each director, return their name and the average `rating` of all `REVIEWED` relationships on movies they directed. Keep only directors with at least 1 reviewed movie. Sort by average rating descending.

Answer.

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie)<-[r:REVIEWED]-(:Person)
WITH d, avg(r.rating) AS avgRating, count(r) AS nReviews
RETURN d.name AS director, avgRating, nReviews
ORDER BY avgRating DESC, director;
```

Explanation. Two patterns sharing the `(m:Movie)` node join directors to reviewers of their films. `r.rating` lives on the `REVIEWED` edge, so we bind `r`. Implicit grouping on `d` makes this a per-director average. `count(r)` adds a useful denominator check.

Part C — Conceptual / Short-Answer

C.1 Translating and comparing across models

Q28. SQL → MQL translation

Prompt. Translate the following SQL into an MQL pipeline on the `classicmodels.orders` collection.

```
SELECT   status, COUNT(*) AS n_orders
FROM     orders
WHERE    orderDate >= '2004-01-01'
GROUP BY status
HAVING   COUNT(*) >= 20
ORDER BY n_orders DESC;
```

Answer.

```
pipeline = [
    { "$match":  { "orderDate": { "$gte": "2004-01-01" } } },
    { "$group":  { "_id": "$status", "n_orders": { "$sum": 1 } } },
    { "$match":  { "n_orders": { "$gte": 20 } } },
    { "$project":{ "_id": 0, "status": "$_id", "n_orders": 1 } },
    { "$sort":   { "n_orders": -1 } },
]
result = list(db["orders"].aggregate(pipeline))
```

Explanation. Note the **two `$match` stages**: one before `$group` (SQL's `WHERE`) and one after (SQL's `HAVING`). Ordering matters for both correctness and performance — placing `$match` before `$group` lets the engine skip documents early.

Q29. Why is `$lookup` + `$unwind` the canonical join?

Prompt. Briefly explain why `$lookup` is almost always followed by `$unwind`, and when you would **not** want to `$unwind`.

Answer & Explanation.

`$lookup` always writes its matches into a **new array field**, regardless of the join cardinality. For a 1-to-1 equi-join (e.g. `orders → customers`), the array is length 1 and downstream stages awkwardly have to write `$cust.customerName` with zero `$cust[0]` index notation. `$unwind "$cust"` collapses the length-1 array into a plain subdocument, after which dot notation works naturally.

Two cases where you keep it as an array:

1. **1-to-many** joins where you want to report *all* matches per parent (e.g. every order for a customer) — `$unwind` would duplicate the parent per child.
2. **LEFT-OUTER semantics** where you need to preserve parents with **no match**. Then use `$unwind: { path: "$cust", preserveNullAndEmptyArrays: true }`.

Q30. `$count` stage vs. `$sum: 1` inside `$group`

Prompt. Give one situation where you must use the `$count` **stage**, and one where you must use `{ "$sum": 1 }` inside `$group`. What would go wrong if you swapped them?

Answer & Explanation.

- **`$count` stage** is used when you want the total number of documents flowing into this stage as the **final** output, for example counting matches after `$match`:

  ```
  [ { $match: { "address.country": "USA" } }, { $count: "n" } ]
  // → [ { n: 36 } ]
  ```

  Swapping in `{ $group: { _id: null, n: { $sum: 1 } } }` works but outputs `{ _id: null, n: 36 }`, which carries a useless `_id`.

- **`{ $sum: 1 }` inside `$group`** is mandatory when you want a per-bucket count:

  ```
  { $group: { _id: "$status", n: { $sum: 1 } } }
  ```

  You **cannot** replace this with `$count`, because `$count` is a terminal stage that produces only a single document. It has no notion of grouping.

The mnemonic: `$count` counts **documents**; `$sum: 1` counts **rows in a bucket**.

Q31. Cypher directionality — why `-[:DIRECTED]->` ≠ `<-[:DIRECTED]-`

Prompt. Explain briefly why `MATCH (p:Person)-[:DIRECTED]->(m:Movie)` and `MATCH (p:Person)<-[:DIRECTED]-(m:Movie)` produce **different** result sets in the Movie graph, and when you would write `-[:DIRECTED]-` (no arrow).

Answer & Explanation.

Every relationship in Neo4j has a direction chosen at insert time. In the Movie graph all `DIRECTED` edges go **Person → Movie**.

- `(p:Person)-[:DIRECTED]->(m:Movie)` matches the direction of the data and returns the expected director/movie rows.
- `(p:Person)<-[:DIRECTED]-(m:Movie)` asks for edges that go **Movie → Person**. None exist, so the result is **empty**.
- The undirected form `(p)-[:DIRECTED]-(m)` matches edges **in either direction**. Use it when the domain is naturally symmetric (e.g. `FOLLOWS` viewed as a general "connection"), or when you explicitly want to ignore orientation.

Direction is not cosmetic — it is part of the pattern's identity, and getting it wrong is the single most common Cypher bug.

Last-minute checklist

- [ ] I can write an MQL pipeline using only `$match, $project, $sort, $limit, $unwind, $group, $lookup, $count`.
- [ ] I know when to `$unwind` after `$lookup` and when to use `preserveNullAndEmptyArrays`.
- [ ] I can compute order revenue with `$unwind` + `$group` + `$sum { $multiply: [...] }`.
- [ ] I can do anti-joins via `$lookup` + `$match: { arr: { $size: 0 } }`.
- [ ] I can translate SQL `WHERE/GROUP BY/HAVING/ORDER BY/LIMIT` into MQL.
- [ ] I can write Cypher triangle patterns like `(a)-[:R1]->(m)<-[:R2]-(b)` and understand arrow direction.
- [ ] I can access relationship properties by binding the edge variable: `-[r:ACTED_IN]-> ... r.roles`.
- [ ] I can use `WITH` to emulate `HAVING` (filter on an aggregate in Cypher).
- [ ] I can use `UNWIND list AS x` and `shortestPath((a)-[*]-(b))`.

Good luck!